

Kadane's Algorithm

Finding the maximum sum contiguous subarray — in plain English, with a live walkthrough and every question variation you'll actually see.

The Problem Maximum Subarray Sum

You're given an array of integers — it can have both positive and negative numbers. You need to find a **contiguous** part of the array (a subarray — no skipping elements) whose sum is the largest possible.

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

```
Best subarray: [4, -1, 2, 1] → Sum = 6
```

Note the word **contiguous** — elements must be next to each other. You can't pick and choose randomly from anywhere in the array. That's a different problem (subsequence).

Think Of It Like This

REAL-LIFE ANALOGY

You're a Zomato delivery partner. Each day you note your **net earning** — some days it's profit, some days it's a loss (fuel cost more than what you earned). You want to find the best **continuous streak of days** where your total profit was the highest.

Every day you ask one question: *"Should I continue my current streak, or is today bad enough that I should just start a fresh streak from today?"* That one question — asked at every single day — is the entire algorithm.

Why Not Just Brute Force? $O(n^2)$

The obvious way: try **every** possible subarray, sum it up, and keep the maximum.

```
int maxSubArrayBrute(vector<int>& nums) {
    int n = nums.size(), best = INT_MIN;
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum += nums[j];
            best = max(best, sum);
        }
    }
    return best;
}
```

This works, but it's $O(n^2)$ — for every starting point `i`, you re-scan forward. Kadane's algorithm does the same job in a **single pass**, $O(n)$, by being smart about reusing the previous sum instead of recomputing it.

The Two Golden Rules

RULE 1 – `currSum` (your "today's mood")

At every index, decide: extend the previous streak, or restart fresh from here? Just take whichever is bigger:

```
currSum = max(nums[i], currSum + nums[i]);
```

If `currSum` already became negative, adding it only drags you down — so it's better to drop it and restart from the current element.

RULE 2 – `maxSum` (your "personal best ever")

The best streak might have already ended a few days back. So separately, keep a variable that just remembers the highest `currSum` you've ever seen:

```
maxSum = max(maxSum, currSum);
```

That's it. Two variables, one loop, one comparison each. `currSum` can go up and down — `maxSum` only ever goes up.

Watch It Run final state

Walking through the algorithm on `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` — shown below after the loop has fully finished. Teal = the best window found, `[4, -1, 2, 1]`.

-2 1 -3 4 -1 2 1 -5 4

CURRSUM (LAST VALUE)

5

MAXSUM (PERSONAL BEST)

6

Loop finished. Best streak = indices 3-6 → sum = 6. (Open the .html version of these notes for the interactive, clickable step-through.)

Same Example — Quick Reference Table

i	nums[i]	currSum	maxSum
0	-2	-2	-2
1	1	1	1
2	-3	-2	1
3	4	4	4
4	-1	3	4
5	2	5	5
6	1	6	6
7	-5	1	6
8	4	5	6

Answer: **maxSum = 6**, achieved by subarray [4, -1, 2, 1] (indices 3-6).

Full C++ Code 0(n) time, 0(1) space

```
int maxSubArray(vector<int>& nums) {
    int currSum = nums[0];
    int maxSum = nums[0];

    for (int i = 1; i < nums.size(); i++) {
        currSum = max(nums[i], currSum + nums[i]); // extend or restart
        maxSum = max(maxSum, currSum);           // update best-ever
    }
    return maxSum;
}
```

If you also need to print the subarray (not just the sum), track `start`, `end`, and `tempStart` alongside:

```
int currSum = nums[0], maxSum = nums[0];
int start = 0, end = 0, tempStart = 0;

for (int i = 1; i < nums.size(); i++) {
    if (nums[i] > currSum + nums[i]) {
        currSum = nums[i];
        tempStart = i;           // new window starts here
    } else {
        currSum += nums[i];
    }
    if (currSum > maxSum) {
        maxSum = currSum;
        start = tempStart;
        end = i;
    }
}
// answer subarray = nums[start..end]
```

How To Recognize a Kadane's Question

In an exam or interview, the question won't say "use Kadane's algorithm." You have to spot it. Look for these signals:

→ The word **"subarray"** (contiguous), not "subsequence" — and you're asked to maximize/minimize a **sum**.

→ The array has a **mix of positive and negative numbers** — that's exactly when greedy "extend or restart" logic matters.

→ Phrasing like *"continuous days," "best streak," "longest profitable run"* — real-world dressing of the same idea.

→ If it says **"circular array"** — still Kadane, but with one extra trick (see variations below).

→ If it says "**product**" instead of "sum" — still Kadane-family, but you track two running values instead of one.

Trap to avoid: If elements can be picked **non-contiguously** (i.e. it says "subsequence," or you're allowed to skip elements, like House Robber), that is **NOT** Kadane's — that's a different DP pattern entirely.

Question Variations You'll See

1. Classic Maximum Subarray

LeetCode 53

Direct application — no twist.

Exactly what's explained above. Just run plain Kadane's.

2. All Elements Negative

Common edge case

Watch your initialization!

If `nums = [-3, -8, -1, -5]`, the answer must still be `-1` (you must pick at least one element). The classic mistake is initializing `maxSum = 0` — that silently gives the wrong answer. Always initialize `currSum` and `maxSum` with `nums[0]`, not `0`.

3. Maximum Sum Circular Subarray

LeetCode 918

Array wraps around (last element connects back to first).

Two cases: either the best subarray is normal (no wraparound) — plain Kadane handles it. Or it wraps around the end — in that case, the best wraparound sum = `totalSum - minSubarraySum`, where `minSubarraySum` is found by running an "inverted" Kadane's (tracking minimum instead of maximum). Final answer = `max(normalKadane, totalSum - minKadane)`. Special case: if all numbers are negative, the wraparound trick gives `0` (an empty array), which is invalid — so just return the normal Kadane's result in that case.

4. Maximum Product Subarray

LeetCode 152

Sum → Product. Negatives flip the logic.

With multiplication, a negative number can turn the smallest (most negative) product into the largest one. So you must track **both** a running max and a running min at every step, and swap them when you hit a negative number:

```
if (nums[i] < 0) swap(maxProd, minProd);
maxProd = max(nums[i], maxProd * nums[i]);
minProd = min(nums[i], minProd * nums[i]);
result = max(result, maxProd);
```

5. Maximum Sum Rectangle in a 2D Matrix

GFG / LeetCode 363

Kadane's extended to 2 dimensions.

Fix a pair of left and right columns, compress every row between them into a single 1D array (row sums), then run plain Kadane's on that 1D array to get the best "height" for that column pair. Repeat for all column pairs and keep the best result. Time: $O(\text{cols}^2 \times \text{rows})$.

6. Maximum Subarray Sum With At Most One Deletion

LeetCode 1186

You're allowed to delete exactly one element to improve the sum.

Track two running states instead of one: `noDelete[i]` = best sum ending at `i` with no deletion used (plain Kadane), and `oneDelete[i]` = best sum ending at `i` with exactly one deletion used so far (either delete `nums[i]` and carry forward `noDelete[i-1]`, or don't delete and carry forward `oneDelete[i-1] + nums[i]`).

7. Print the Actual Subarray

Follow-up question

Not just the sum — the actual elements.

Shown in the code section above — track `start`, `end`, and `tempStart` indices alongside `currSum/maxSum`.

Common Mistakes

Wrong init: Setting `maxSum = 0` instead of `nums[0]` — breaks on all-negative arrays.

Subarray vs subsequence: Applying Kadane's when the problem actually allows skipping elements (non-contiguous) — that needs a different DP, not Kadane's.

Forgetting indices: If asked to print the subarray, forgetting to update `start/end` only when `maxSum` actually improves (not every iteration).

Circular trap: Forgetting the "all negative" special case in the circular subarray variation — the wraparound formula breaks silently.

Practice List

- LeetCode 53 — Maximum Subarray
- LeetCode 918 — Maximum Sum Circular Subarray
- LeetCode 152 — Maximum Product Subarray
- LeetCode 1186 — Maximum Subarray Sum with One Deletion
- GFG — Maximum Sum Rectangle in a 2D Matrix

